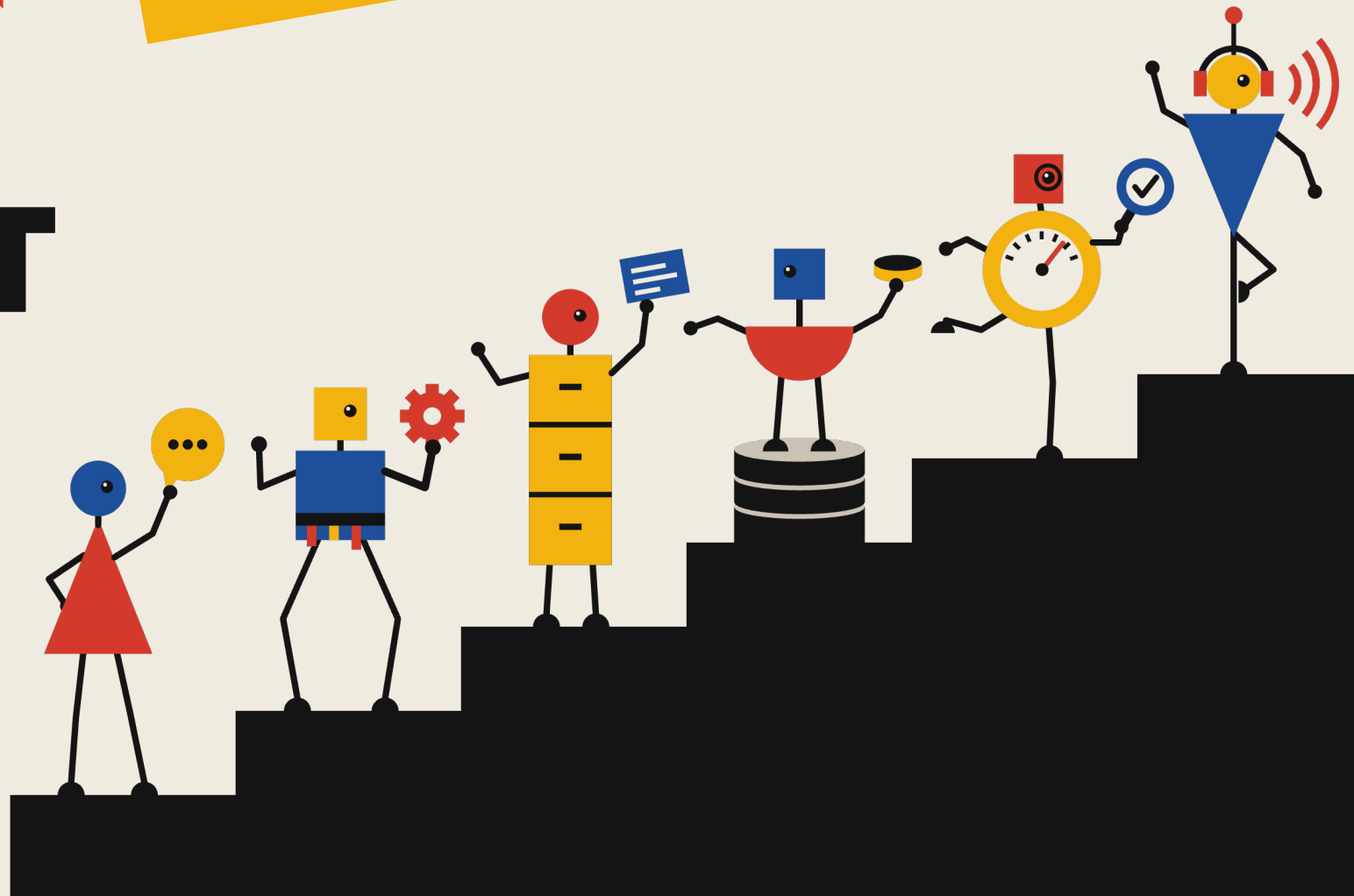
BUILD NEXT-GEN AI APPS WITH .NET

David Ortinau

Techorama NL | 28 October 2026

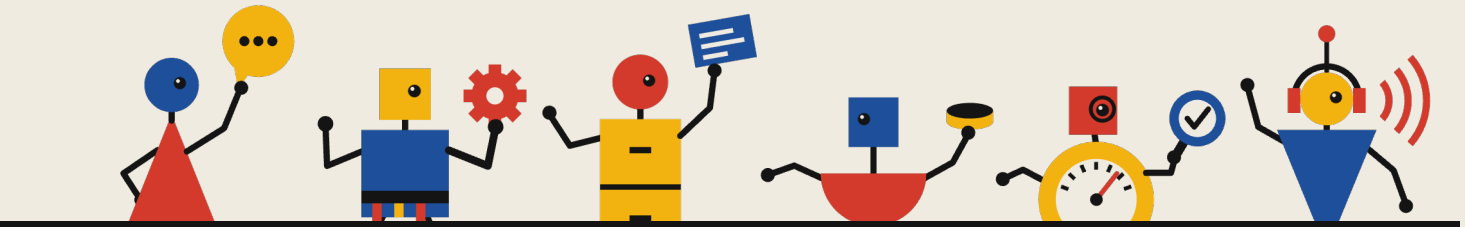


DAVID ORTINAU

Principal Product Manager for .NET at Microsoft

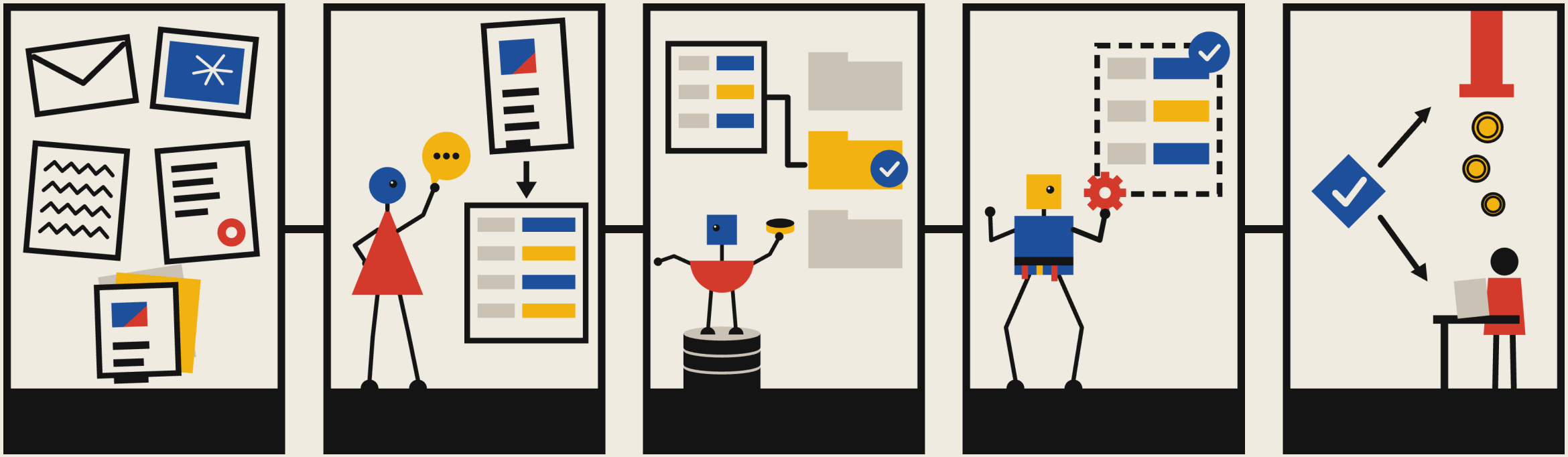
- .NET AI, Aspire, Blazor, MAUI, & Security
- Building apps since the 90s
- Sentence Studio is my own app

@davidortinau





ONE CLAIM,
ANY FORMAT



Arrives **Read** **Linked** **Proposed** **Decided**

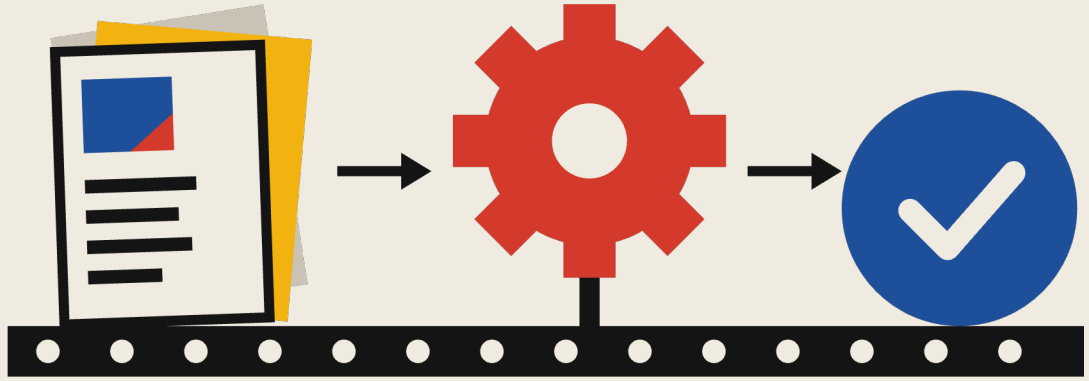
"The hard part was not only extracting data, but also validating it, connecting it to the right claim context..."

Pavel Lukeš, Head of IT, Direct Pojišťovna | Microsoft customer story

USEFUL WORK,
NOT MORE OUTPUT

15 min

about **2** min



Windshield-claim handling

Direct Pojišťovna | customer-reported

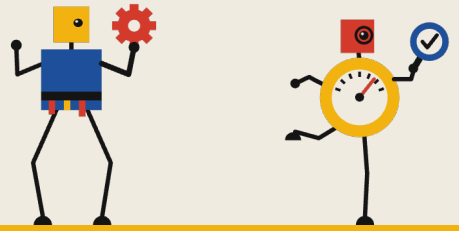
WHAT DIRECT BUILT ON AZURE

Direct Pojišťovna with BigHub | customer-reported



Forms, photos, invoices, emails

Azure Document Intelligence
reads and extracts



Agents built on Microsoft Foundry
check and route

Payout





People
the complex cases

87%
less handling time

60%
of claims fully automated

24/7
always running

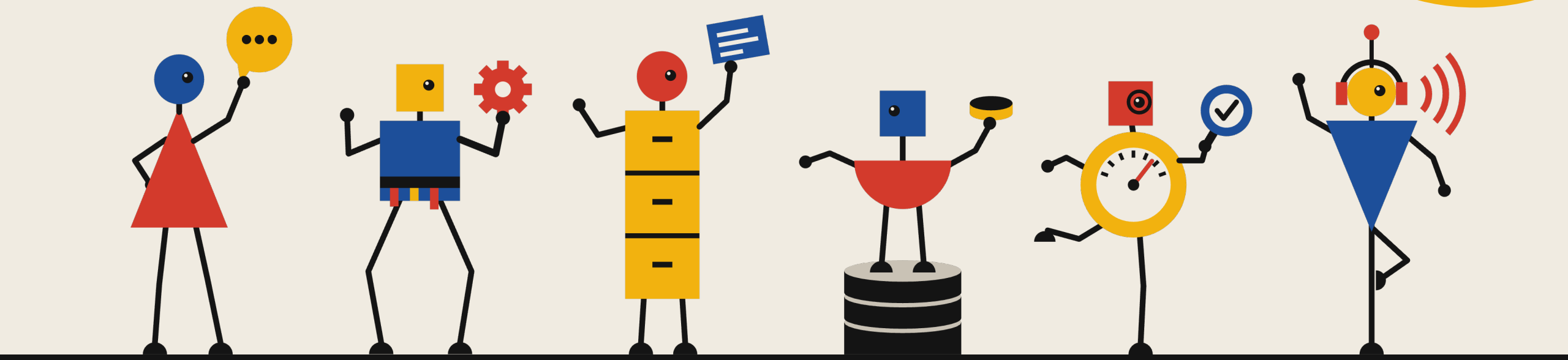
100%
text-extraction precision in the pilot

WHAT GOOD LOOKS LIKE



1. Stay inside the task
2. Choose input and output separately
3. Deliver something people can check and use

WHAT .NET GIVES YOU



MCP | Foundry and Foundry Local | Aspire

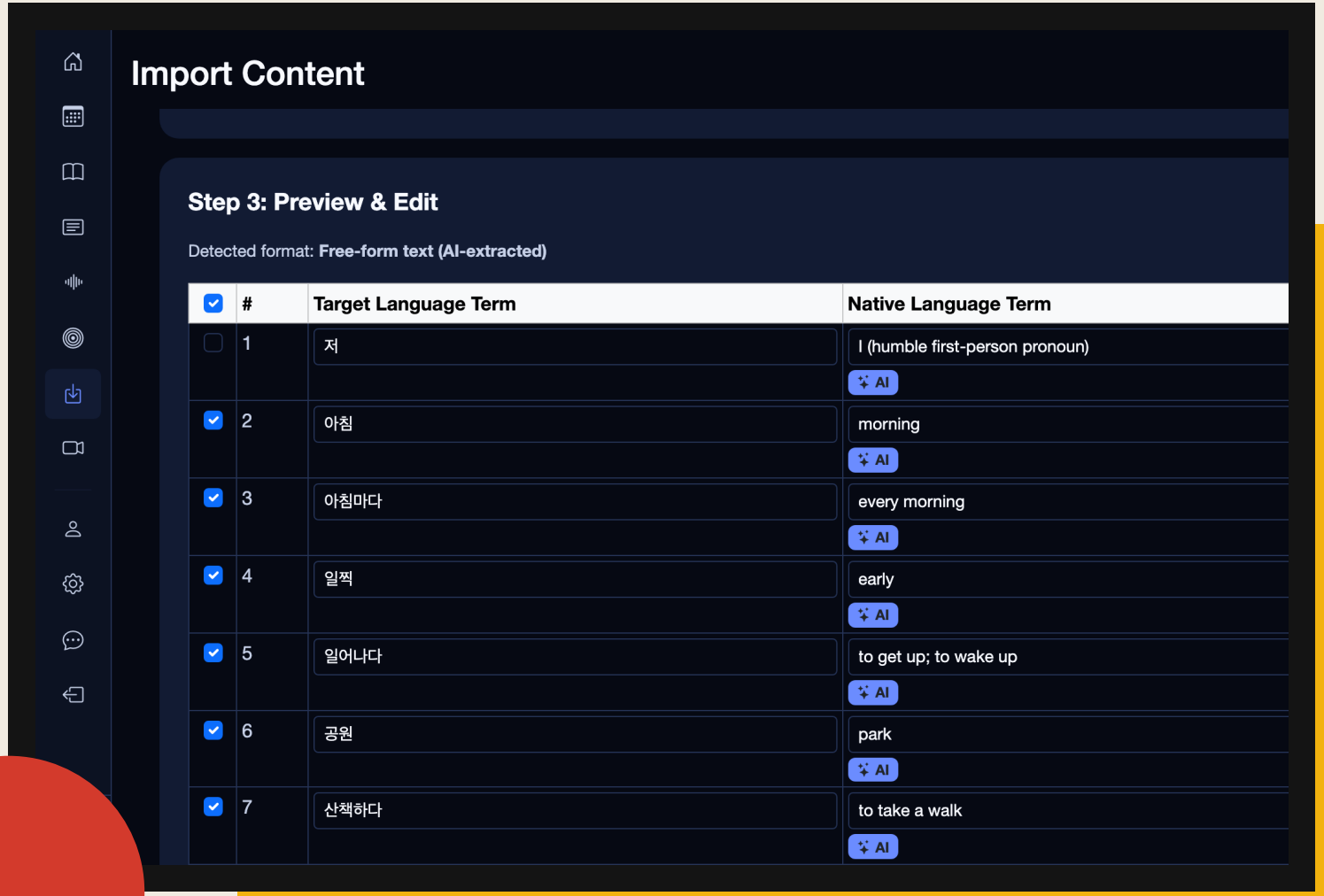
Talk to models	Define agents	Build memory	Ground in data	Evaluate	Talk in real time
MEAI	Agent Framework	Agent Framework	MEDI + MEVD	MEAI Evaluation	MEAI Realtime

ON DUTY

DEMO 1

SENTENCE STUDIO

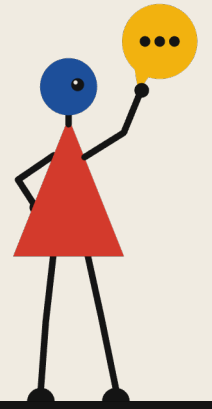
My material → my practice



ONE INTERFACE, ANY MODEL

ON DUTY





Simplified from Sentence Studio

```
services.AddKeyedSingleton<IChatClient>("fast",
    (sp, _) => openAI.GetChatClient(fastModel).AsIChatClient());
```

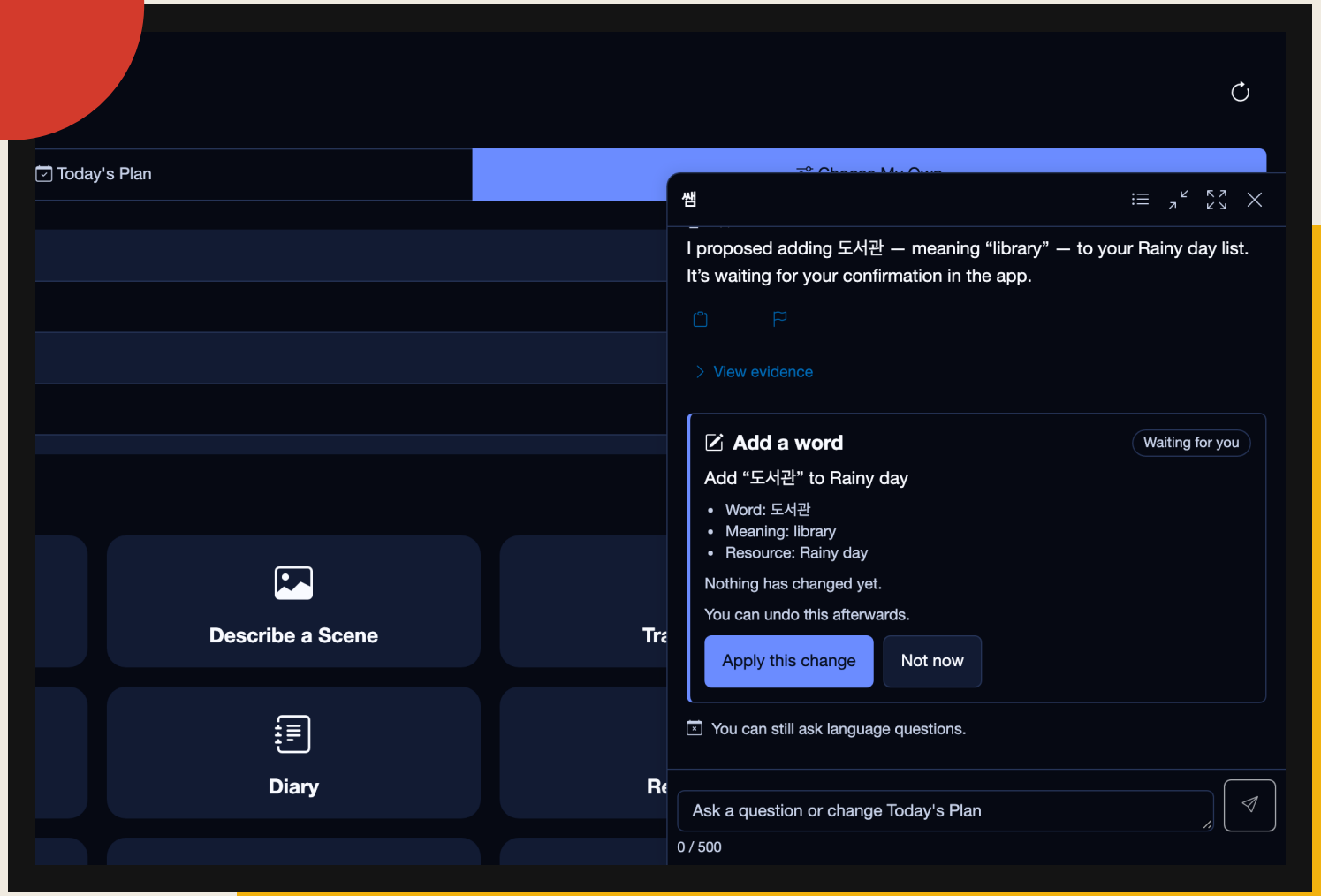
```
var draft = await chat.GetResponseAsync<FreeTextVocabularyExtractionResponse>(messages);
```

ON DUTY

DEMO 2

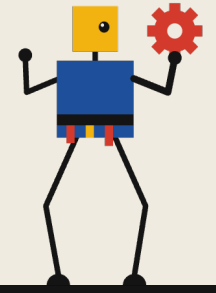
MEET SAM

A coach that knows what you studied



MODELS PROPOSE.
APPS DECIDE.

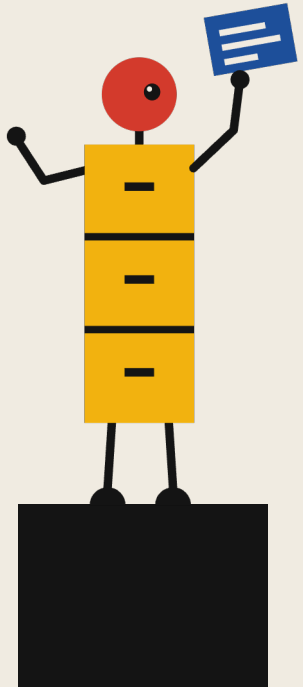
ON DUTY



Agent = model + instructions + tools + session

Simplified from Sentence Studio's CoachAgentFactory

```
AIAgent coach = chatClient.AsAIAgent(new ChatClientAgentOptions
{
    ChatOptions = new() { Instructions = instructions, Tools = tools }
});
AgentSession session = await coach.CreateSessionAsync();
var reply = await coach.RunAsync(message, session);
```



Memory the learner can see, approve, and forget

candidate → approved → edited or forgotten

GROUND IN DATA

ON DUTY

Your app's records: typed tools

Tools

owner-scoped reads

Your documents: ingest → embed → retrieve

MEDI (preview)

read, chunk, enrich

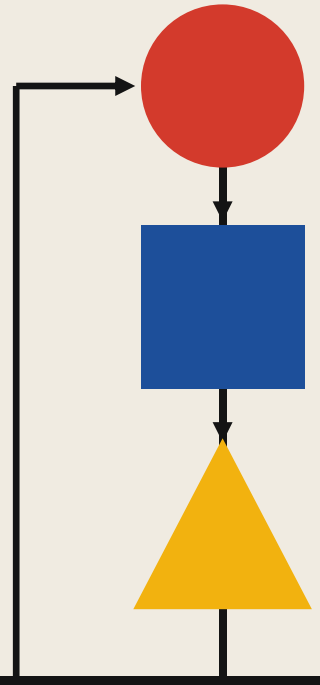
MEVD

store, retrieve

Agent



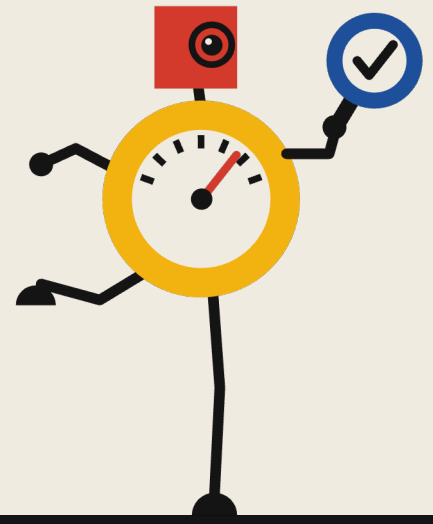
EVALUATE TO KEEP IMPROVING



Before release: evaluators in your tests

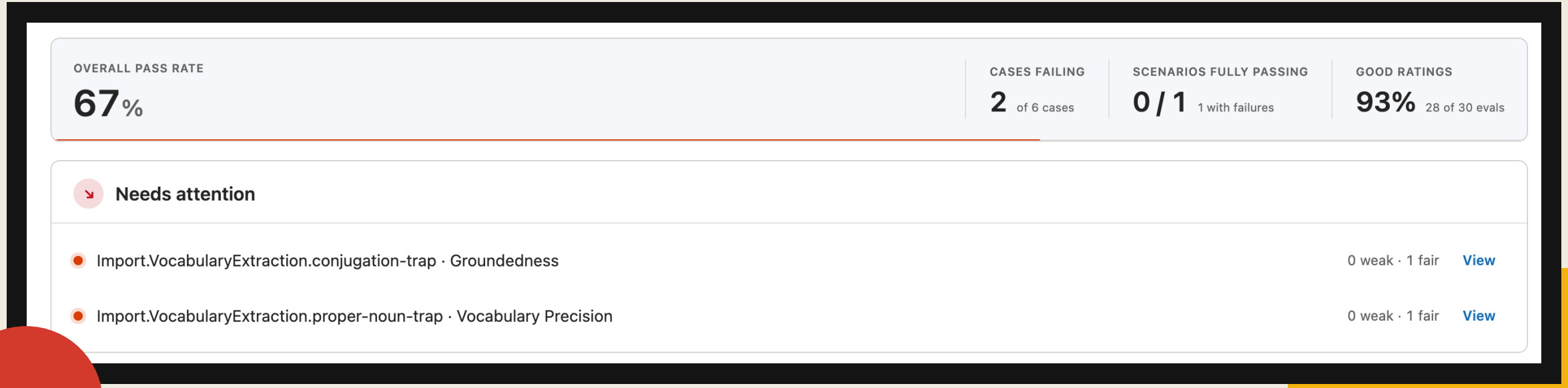
In production: count ungrounded claims

Compare pipelines on the same tools



MEASURE THE WHOLE JOB

ON DUTY



SHORT DEMO | 6 SYNTHETIC FIXTURES, DEMO SCALE

EVALUATION REPORT

TALK IN REAL TIME

ON DUTY



Same pipeline: tools, telemetry, logging

```
IChatClient chat = chatClient
    .AsBuilder()
    .UseFunctionInvocation()
    .UseOpenTelemetry()
    .UseLogging()
    .Build();
```

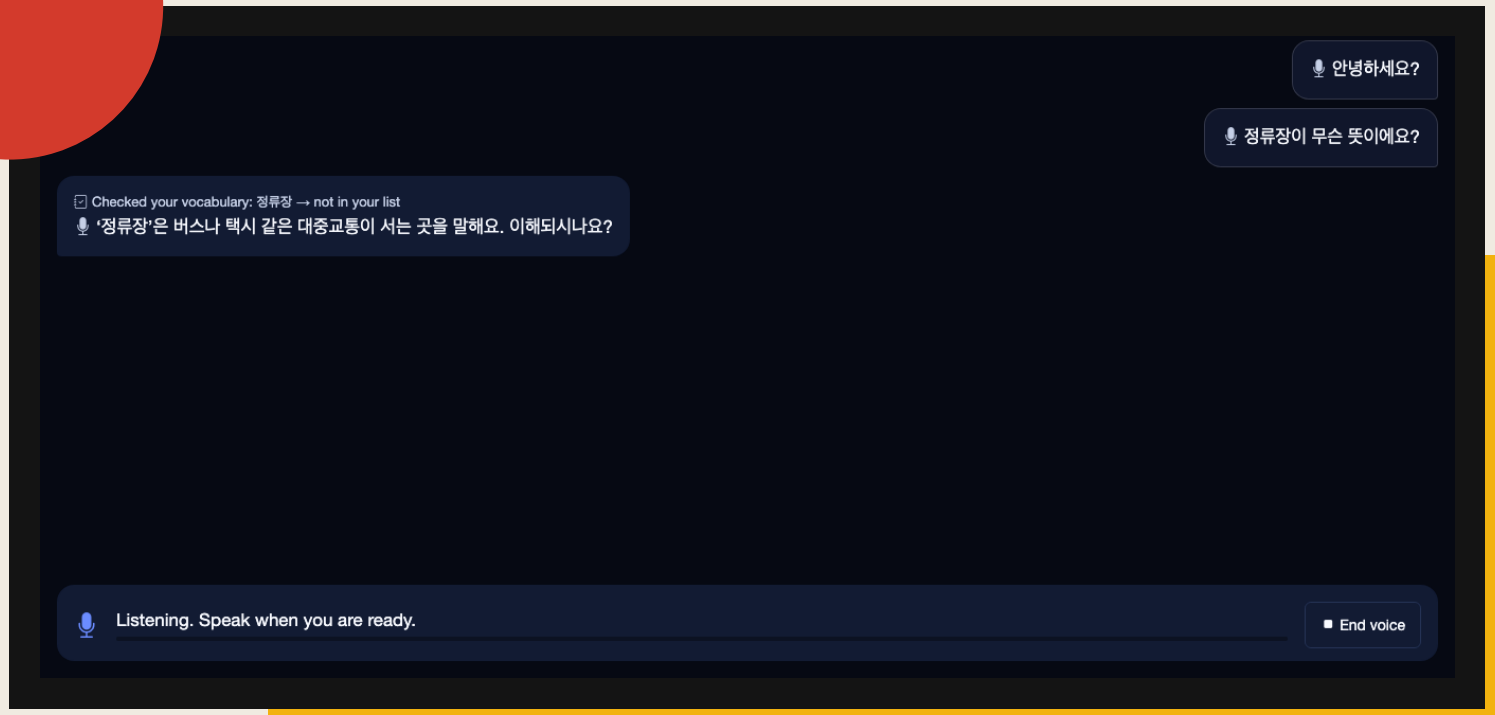
```
IRealtimeClient voice = realtimeClient
    .AsBuilder()
    .UseFunctionInvocation()
    .UseOpenTelemetry()
    .UseLogging()
    .Build();
```

Experimental (MEAI001)

DEMO 3

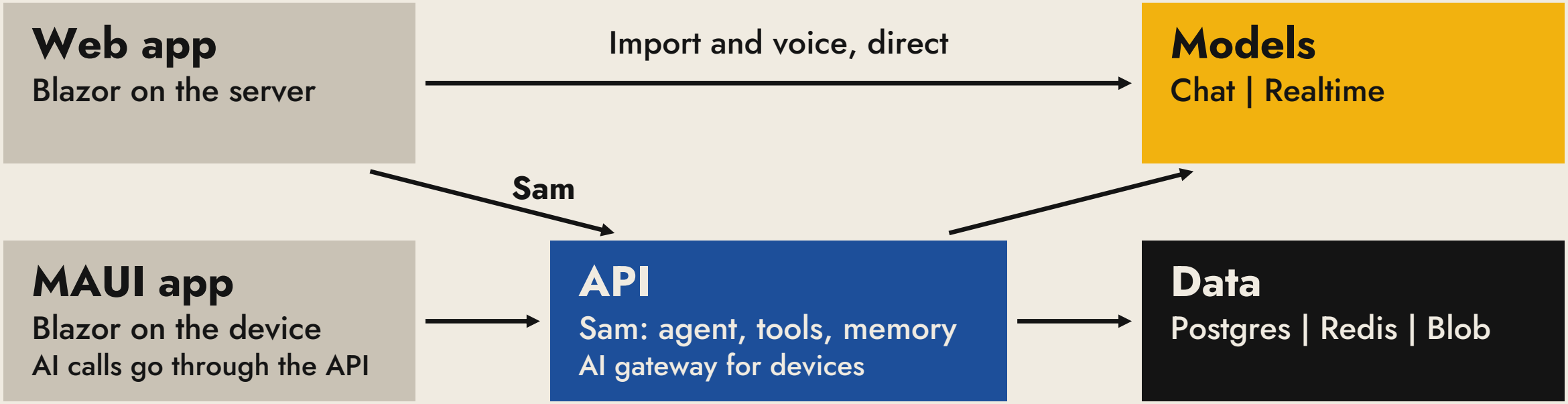
TALK IN REAL TIME

Voice in the Conversation activity



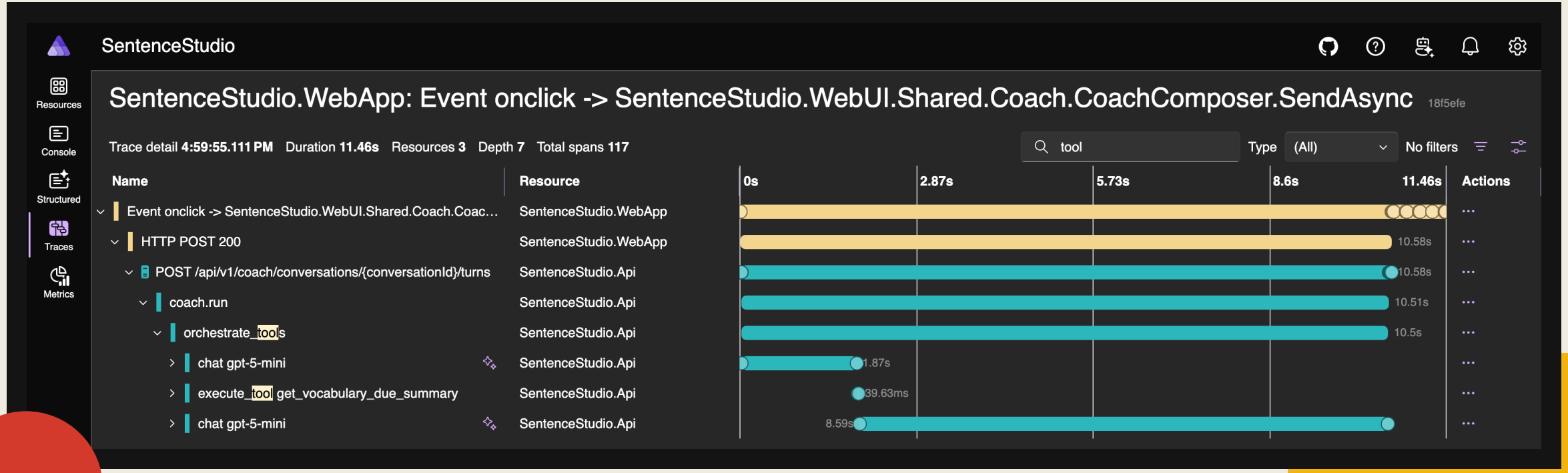
HOW IT FITS TOGETHER

ON DUTY



ASPIRE RUNS IT AND TRACES IT

ON DUTY



DEMO RUN

ASPIRE DASHBOARD

One Sam turn: model, tool, model

WHERE .NET AI IS HEADING

NOW

MEAI **10.11**

Evaluation **10.11**

MEVD **10.11**

Agent Framework **1.25**

Harness Agent, Workflows

MCP C# SDK **2.3**

Foundry Local

Aspire

NEXT

.NET 11 GA
November 2026

Data ingestion
(preview)

Realtime
(experimental)

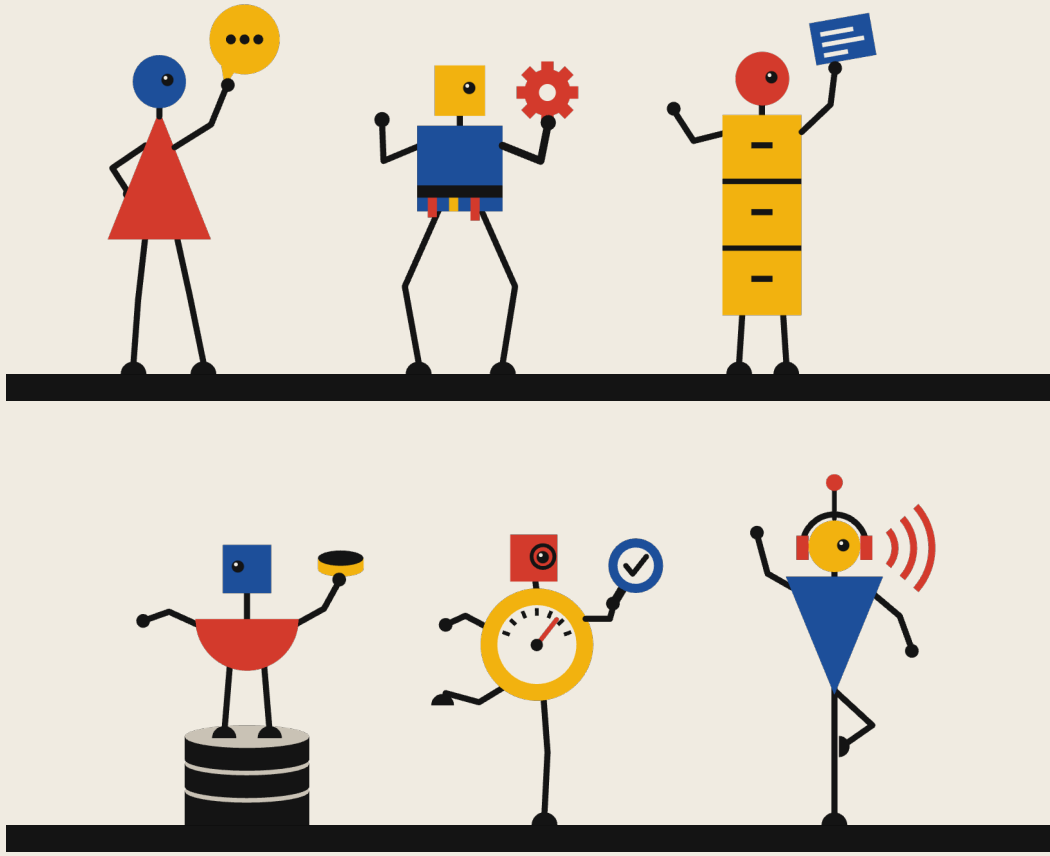
LATER

[illegible]

PAST THE <BLINK> TAG



1. Start with the job
2. Use the smallest block that finishes it
3. Models propose. Apps decide.
4. Measure the whole job



QUESTIONS

What would your users finish?

David Ortinau



SpreaView

Review my Session